

Explication et guide KiCad — TFC Système Multicanal

Notions de base + pas-à-pas pour dessiner le schéma

Document préparé pour Samuel Mumbere Mathe — ULPGL

Juillet 2026

Table des matières

Introduction	1
Partie 1 — Les bases de KiCad et de l'électronique qu'il faut connaître	1
C'est quoi KiCad ?	1
C'est quoi un "symbole" ?	2
C'est quoi une "empreinte" (footprint) ?	2
C'est quoi un "netlist" ?	2
C'est quoi l'ERC (Electrical Rules Check) ?	2
Zone haute tension / basse tension, isolation galvanique	2
Partie 2 — Guide pas-à-pas : dessiner le schéma	3
Étape 0 — Lancer KiCad	3
Étape 1 — Créer le projet	3
Étape 2 — Format de feuille	3
Étape 3 — Placer les composants	3
Étape 4 — Séparer les zones HT et BT (§2.5.1 du mémoire)	4
Étape 5 — Câblage : reprends le Tableau 2.2 (corrigé) du mémoire	5
Étape 6 — Vérifier avant de continuer	5
Étape 7 — Sauvegarder correctement	5
Et après ?	5

Introduction

Ce document a deux parties : d'abord les notions de base pour comprendre ce qu'on fait dans KiCad (utile même si tu ne dessines rien toi-même, pour répondre au jury), puis le guide pratique, pas-à-pas, pour redessiner le schéma de ton système à 8 canaux.

Partie 1 — Les bases de KiCad et de l'électronique qu'il faut connaître

C'est quoi KiCad ?

KiCad est un logiciel gratuit de conception électronique (on dit "EDA" : Electronic Design Automation). Il sert à transformer une idée de circuit en un vrai fichier de fabrication qu'une usine peut lire pour produire un circuit imprimé (PCB). Il y a deux grands outils dans KiCad :

1. **Schematic Editor** (éditeur de schéma) — où on dessine le schéma électrique : quels composants, et comment ils sont reliés entre eux (comme un plan, pas encore un objet physique).
2. **PCB Editor** (éditeur de circuit imprimé) — où on transforme ce schéma en un vrai dessin de plaque, avec l’emplacement physique de chaque composant et le tracé des pistes de cuivre qui les relient.

Ton mémoire (§2.5) couvre les deux : le schéma (§2.5.1-2.5.2) puis le PCB (§2.5.3-2.5.4). Ce guide se concentre sur le **schéma d’abord** — le PCB est l’étape suivante, une fois le schéma solide (voir le document séparé `explication_pcb.pdf`).

C’est quoi un “symbole” ?

Un symbole, c’est la représentation graphique standardisée d’un composant sur un schéma — un rectangle avec des pattes numérotées pour une puce, un zigzag pour une résistance, etc. Ce n’est **pas** un dessin réaliste du composant, juste une convention universelle pour que n’importe quel ingénieur comprenne le schéma. Chaque symbole est identifié par un nom, rangé dans une **bibliothèque** (un fichier `.kicad_sym` qui contient plusieurs symboles apparentés, par exemple tous les microcontrôleurs Espressif).

C’est quoi une “empreinte” (footprint) ?

Une empreinte, c’est le dessin de l’**emplacement physique réel** des pattes d’un composant sur le circuit imprimé (les petits rectangles de cuivre où on soudera le composant). Un même symbole électrique (par exemple une résistance) peut avoir plusieurs empreintes possibles, selon la taille physique du composant qu’on va réellement souder. On choisit l’empreinte à l’étape PCB, pas à l’étape schéma.

C’est quoi un “netlist” ?

Une fois le schéma terminé, KiCad peut en extraire une liste de toutes les connexions électriques prévues (*“la broche 3 du composant A est reliée à la broche 1 du composant B”,* etc.) — c’est le netlist. C’est ce fichier qui fait le pont entre le schéma et le PCB : KiCad l’utilise pour savoir quelles pistes de cuivre tracer.

C’est quoi l’ERC (Electrical Rules Check) ?

Un vérificateur automatique qui relit ton schéma et signale les erreurs probables : une broche qui n’est reliée à rien, deux sorties reliées directement entre elles (conflit électrique), etc. Toujours le lancer avant de continuer — c’est l’équivalent d’un correcteur orthographique, mais pour un circuit.

Zone haute tension / basse tension, isolation galvanique

Ton système manipule à la fois du 230V (secteur) et du 3.3V/5V (électronique de l’ESP32). Physiquement séparer ces deux zones sur le schéma **et** sur le circuit imprimé (avec une distance de sécurité suffisante entre les pistes) est une exigence de sécurité, pas juste d’organisation — un défaut du côté 230V ne doit jamais pouvoir atteindre la partie que l’utilisateur touche (l’ESP32, le port USB...). C’est exactement ce que rappelle ton mémoire en §1.3.4 et §2.5.1, et que les capteurs ACS712/ZMPT101B assurent physiquement (isolation galvanique = aucune connexion électrique directe entre les deux côtés, seulement un couplage magnétique ou optique).

Partie 2 — Guide pas-à-pas : dessiner le schéma

Tu peux commencer **avant** d’avoir l’ESP32 physique — rien ici ne demande le matériel en main.

État au 17 juillet : d’après ton fichier `multical.kicad_sch`, il te manque un deuxième ULN2003A (tu n’en as qu’un, avec 7 sorties pour 8 relais — voir étape 3 ci-dessous) et le câblage n’est pas encore fait — voir la correction du Tableau 2.2 à l’étape 5 avant de commencer à câbler.

Étape 0 — Lancer KiCad

Deux installations existent sur cette machine : une snap (KiCad 9.0.7, périmée) et une flatpak (KiCad 10.0.4, la version stable actuelle, vérifiée en juillet 2026). Utilise la flatpak :

```
flatpak run org.kicad.KiCad
```

(ou depuis le menu applications — si deux icônes “KiCad” apparaissent, vérifie dans Aide → À propos que tu ouvres bien la 10.0.4)

Étape 1 — Créer le projet

1. **Fichier → Nouveau projet**
2. Nomme-le `multicanal_goma` (évite les espaces/accents dans le nom de fichier — KiCad n’aime pas toujours ça)
3. Sauvegarde-le directement dans : `~/Documents/TFC-Multicanal-Goma/kicad/` (là où se trouve déjà ce guide et le symbole `zmpt101b.kicad_sym`, à côté du firmware et de l’application)
4. Double-clique sur l’icône **Schematic Editor** (feuille bleue) dans la fenêtre du projet

Étape 2 — Format de feuille

Fichier → Propriétés de la feuille → choisis **A3** (le schéma a beaucoup de composants, A4 sera trop serré). Remplis le cartouche (titre, date, ton nom, “Encadreur: Patrick Nzanu Vingi” — retrouvé dans un ancien fichier de travail).

Étape 3 — Placer les composants

Touche **A** (ou icône “Placer un symbole”) ouvre le sélecteur. Tape le nom exact ci-dessous dans la barre de recherche.

Important : un ancien fichier de travail contenait des noms de composants qui n’existent pas réellement dans les bibliothèques KiCad — c’est probablement pour ça qu’il ne s’ouvrait plus. Chaque nom ci-dessous a été vérifié individuellement contre les vraies bibliothèques installées sur ta machine, **dans les deux versions de KiCad présentes** (9.0.7 et 10.0.4 — identiques sur ce point).

Cherche exactement	Bibliothèque réelle	Combien	Où le placer
ESP32-WROOM-32	RF_Module	1	Centre
ACS712xLCTR-20A	Sensor_Current	8	Colonne de gauche, zone BT

Cherche exactement	Bibliothèque réelle	Combien	Où le placer
ULN2003A	Transistor_Array	2	Entre ESP32 et relais (chacun pilote jusqu'à 7 sorties, 2 suffisent pour 8 relais — tu n'en as qu'1 pour l'instant)
Relay_SPDT	Relay	8	Colonne de droite, zone HT (voir note ci-dessous)
HLK-PM01	Converter_ACDC	1	Zone HT → alimentation
AMS1117-3.3	Regulator_Linear	1	Zone BT, juste après le HLK-PM01
D (diode générique)	Device	8	Une en parallèle de chaque relais (roue libre)
R	Device	5	Pull-up strapping pins + limitation courant
C	Device	5	Découplage (près des VCC ACS712 et ESP32)
Conn_01x02_Pin	Connector	9	Un par canal + alimentation secteur

ZMPT101B — pas dans les bibliothèques standard

Ce capteur n'existe dans aucune bibliothèque KiCad officielle (normal, c'est un module très "communauté Arduino", pas un composant industriel référencé). Un symbole personnalisé valide existe déjà dans ~/Documents/TFC-Multicanal-Goma/kicad/zmpt101b.kicad_sym (6 broches : L, N, GND, GND, VCC, OUT — correspond au vrai module), et tu l'as déjà importé avec succès (visible sur ton schéma actuel, U12).

Relais SLA-05VDC-SL-A — pas de symbole exact non plus

Le nom précis "SLA-05VDC-SL-A" (celui des modules relais tout faits pour Arduino) n'a pas de symbole officiel KiCad. Le symbole générique Relay_SPDT (déjà utilisé sur ton schéma pour K1-K8) est fonctionnellement identique — bobine 5V + contact inverseur. Pense à changer le champ **Valeur** de "Relay_SPDT" à "SLA-05VDC-SL-A" dans les propriétés de chaque relais pour que ta nomenclature (BOM) reste exacte.

Étape 4 — Séparer les zones HT et BT (§2.5.1 du mémoire)

C'est le point que ton mémoire promet déjà — respecte-le visuellement dans le schéma, pas juste dans le texte. Sur ton schéma actuel, les 8 relais/diodes sont déjà groupés à droite et l'électronique basse tension à gauche, ce qui va dans le bon sens, mais il manque encore la séparation explicite :

- **Zone haute tension (230V)**, à droite ou en haut de la feuille : arrivée secteur (Conn_01x02_Pin), les 8 relais, le HLK-PM01.
- **Zone basse tension (3.3V/5V)**, à gauche ou en bas : ESP32, les 8 ACS712, l'AMS1117, les ULN2003A.

- Le ZMPT101B et les ACS712 sont la frontière (isolation galvanique, cf. §1.3.4) — dessine une ligne ou une boîte en pointillés entre les deux zones sur la feuille, avec une annotation “ISOLATION GALVANIQUE” comme rappel visuel pour le jury.

Étape 5 — Câblage : reprends le Tableau 2.2 (corrigé) du mémoire

Correction importante : contrairement à ce qui était dit dans une version précédente de ce guide, le symbole ESP32-WROOM-32 que tu utilises (vérifié directement dans la bibliothèque KiCad) **n’a pas non plus** les broches GPIO37/GPIO38 — elles n’existent pas sur ce module, DevKitC ou pas. Il n’y a donc que 6 broches ADC1 réellement utilisables (36, 39, 32, 33, 34, 35) pour 8 canaux. Les canaux 2 et 3 sont déplacés sur ADC2 (GPIO26, GPIO27), sur le même principe déjà utilisé pour le ZMPT101B (lecture pendant les fenêtres où le Wi-Fi est inactif, cf. §1.4.3 du mémoire — désormais 3 signaux à gérer ainsi au lieu d’1, mais aucun composant supplémentaire nécessaire) :

Canal	ACS712 → ESP32	ADC	Relais → ESP32 (via ULN2003A)
1 (bailleur)	GPIO36	ADC1	GPIO4
2	GPIO26	ADC2	GPIO5
3	GPIO27	ADC2	GPIO13
4	GPIO39	ADC1	GPIO14
5	GPIO32	ADC1	GPIO16
6	GPIO33	ADC1	GPIO17
7	GPIO34	ADC1	GPIO18
8	GPIO35	ADC1	GPIO19

ZMPT101B → GPIO25 (ADC2).

Évite ces broches pour toute nouvelle affectation (broches de configuration au démarrage, dites “strapping pins”, déjà notées en §2.5.2 du mémoire) : GPIO0, GPIO2, GPIO12, GPIO15.

Pour tracer un fil : touche **W** (wire), clique départ, clique arrivée. Pour nommer un fil (utile pour les signaux qui se répètent, ex. CANAL1_ACS) : sélectionne-le puis touche **L** (label). Donner le même nom à deux bouts de fil différents les relie électriquement, sans avoir à tracer un trait continu à travers toute la feuille — pratique pour éviter un schéma illisible avec des lignes qui se croisent partout.

Étape 6 — Vérifier avant de continuer

Inspecter → Vérification des règles électriques (ERC) — lance-le, corrige les erreurs (broches non connectées, conflits de type de pin). Ne passe au routage du circuit imprimé (PCB, §2.5.3 du mémoire) qu’une fois l’ERC propre.

Étape 7 — Sauvegarder correctement

Ctrl+S régulièrement. KiCad enregistre automatiquement les définitions complètes des symboles utilisés dans le fichier — tant que tu passes par le vrai logiciel (et pas un script), un fichier cassé comme l’ancien ne peut pas se reproduire.

Et après ?

Une fois le schéma terminé et l’ERC propre, l’étape suivante est le **routage du circuit imprimé** — voir le document séparé `explication_pcb.pdf`, déjà prêt pour cette étape.

Reviens vers moi avec une capture d'écran ou une description si tu bloques sur un placement ou un câblage précis — je peux aussi lire un export PDF du schéma directement si tu me donnes le chemin du fichier.